

Labo 01 — Conteneurs et architecture : Docker, Podman et le noyau Linux

Théorie — ce qu'est réellement un conteneur, qui fait quoi quand vous tapez `podman run`, et pourquoi « Docker » désigne aujourd'hui à la fois un outil et une façon de travailler.

Objectifs

- Savoir dire ce qu'est un conteneur **sans** employer le mot « machine virtuelle légère ».
- Nommer les deux mécanismes du noyau Linux qui rendent les conteneurs possibles.
- Distinguer les acteurs : client, moteur (daemon ou pas), image, registry — chez Docker **et** chez Podman.
- Distinguer une **image** d'un **conteneur** — la confusion la plus coûteuse pour un débutant.
- Lire n'importe quelle commande `docker` / `podman` et en deviner la structure.

1. Une histoire de « ça marche sur ma machine »

Une application ne tourne jamais toute seule. Une API Spring Boot a besoin d'un JRE dans une version précise, de variables d'environnement, d'un certificat, d'un fuseau horaire. Cet ensemble s'appelle l'**environnement d'exécution**, et pendant vingt ans on l'a installé à la main sur des serveurs. Résultat : le poste du développeur et la production n'étaient jamais identiques, et deux applications sur le même serveur se disputaient la même bibliothèque en deux versions.

La première réponse fut la **machine virtuelle** : un ordinateur complet simulé, avec son propre système d'exploitation, par application. Ça marche — au prix de plusieurs Go de disque, de RAM réservée et d'un démarrage en minutes, pour faire tourner *un* processus.

→ HISTOIRE

En mars 2013, Solomon Hykes présente Docker en cinq minutes à la conférence PyCon. Rien n'est nouveau techniquement : *namespaces* et *cgroups* existent dans Linux depuis 2008, LXC les utilise déjà. Ce que Docker invente, c'est l'**emballage** : une image que l'on construit, publie et lance en une commande. En 2015, Docker cède le format d'image et le runtime à l'**OCI** (*Open Container Initiative*) : depuis, une image est un standard que n'importe quel outil peut faire tourner. Podman (Red Hat, 2018) est né de cette ouverture.

Le conteneur est la seconde réponse : **on isole le processus, pas la machine.**

2. Ce qu'est un conteneur

À RETENIR

Un conteneur est un **processus ordinaire** de votre machine Linux, à qui le noyau ment sur ce qu'il peut voir et sur ce qu'il peut consommer.

Il n'y a pas de système d'exploitation dans un conteneur. Pas d'émulation. Si vous lancez un conteneur `nginx`, il existe sur votre hôte un vrai processus `nginx`, visible dans `ps aux`, exécuté par le **même noyau Linux** que tout le reste. Ce qui change, c'est ce que ce processus perçoit du monde. Deux mécanismes du noyau font ce travail.

→ LINUX

Le **noyau** (*kernel*) est la partie du système qui parle au matériel et arbitre entre les programmes : il crée les processus, leur donne de la mémoire, du temps CPU, l'accès aux fichiers et au réseau. Tout ce qui est « au-dessus » — `bash`, `ls`, `java`, `nginx` — s'appelle l'**espace utilisateur** (*userland*). Un programme ne touche jamais le matériel directement : il demande au noyau via des **appels système** (`open`, `read`, `fork` ...). C'est cette frontière que les conteneurs exploitent.

Les namespaces (espaces de noms) — l'isolation de la vue. Le noyau donne à un processus une vue partielle et privée de certaines ressources :

Namespace	Ce qu'il isole	Conséquence visible
<code>pid</code>	Les identifiants de processus	Dans le conteneur, votre application est le PID 1 et ne voit rien d'autre
<code>net</code>	Interfaces, ports, routes	Le conteneur a son propre port 8080, distinct de celui de l'hôte
<code>mnt</code>	Les points de montage	Le conteneur voit son propre <code>/</code>
<code>uts</code>	Le nom d'hôte	<code>hostname</code> renvoie l'identifiant du conteneur
<code>ipc</code>	Les communications inter-processus	Pas de mémoire partagée avec les voisins
<code>user</code>	Les UID/GID	Un <code>root</code> dans le conteneur peut n'être qu'un utilisateur banal sur l'hôte — c'est le cœur de Podman rootless

Les cgroups (control groups) — la limitation des ressources. Les namespaces cachent, les cgroups plafonnent : « ce groupe de processus n'aura pas plus de 512 Mo de RAM ni de 1,5 cœur ». C'est ce qui empêche un conteneur qui part en vrille d'emporter le serveur avec lui.

Conteneur ou VM ?

	Machine virtuelle	Conteneur
Isole	Un ordinateur entier (matériel virtualisé)	Un ou plusieurs processus
Noyau	Le sien	Celui de l'hôte, partagé
Démarrage	Secondes à minutes	Millisecondes
Poids typique	Plusieurs Go	Quelques dizaines de Mo
Frontière de sécurité	Forte (hyperviseur)	Plus faible (un seul noyau à compromettre)

→ WINDOWS / WSL

« Le conteneur partage le noyau de l'hôte » a une conséquence : un conteneur Linux ne tourne **que** sur un noyau Linux. Sur Windows, c'est **WSL 2** (*Windows Subsystem for Linux*) qui le fournit : une VM très légère, gérée par Hyper-V, qui démarre en une seconde, partage la RAM avec Windows et fait tourner un vrai noyau Linux compilé par Microsoft. Votre Podman s'exécute *dans* cette distribution Ubuntu ; vos conteneurs sont des processus de cette VM, pas de Windows. Docker Desktop et Podman Desktop font la même chose en coulisses : ils créent leur propre distribution WSL.

3. Image et conteneur

C'est la distinction fondamentale de toute la formation.

Une **image** est un modèle **en lecture seule** : un système de fichiers figé (le JRE, votre JAR, les bibliothèques) plus des métadonnées (quelle commande lancer, quelles variables, quel utilisateur, quel port). Une image ne s'exécute pas, ne consomme pas de CPU, ne « tourne » pas. Elle est inerte et **immuable**.

Un **conteneur** est une *instance en cours d'exécution* d'une image : l'image, plus une fine couche d'écriture propre à cette instance, plus un processus vivant.

→ JAVA

L'analogie la plus juste vient de la programmation objet : l'image est la **classe**, le conteneur est l'**objet** (`new`). On instancie vingt conteneurs de la même image ; ils partagent le même contenu en lecture seule et ont chacun leur état privé. Et comme un objet, un conteneur se détruit sans que la classe ne bouge.

À RETENIR

Tout ce que votre application écrit dans un conteneur va dans cette couche d'écriture, **détruite avec le conteneur**. C'est voulu : un conteneur est jetable. La persistance est le sujet du labo 06.

Une image est faite de **couches** (*layers*) empilées, une par étape de construction ; dix images parties du même `eclipse-temurin:21-jre` ne stockent cette base qu'une fois (labo 02).

4. L'architecture : qui fait quoi

Ici, Docker et Podman divergent — et c'est la raison d'être de Podman.

DOCKER	<code>docker</code> (client) —HTTP/socket→ <code>dockerd</code> (daemon, root) → <code>containerd</code> → <code>runc</code>
PODMAN	<code>podman</code> (votre utilisateur) —fork/exec→ <code>conmon</code> → <code>crun</code> (aucun daemon)
	les deux —pull→ Registry (Docker Hub, Harbor, ECR...)

Docker est une architecture **client / serveur**. Le binaire `docker` ne fait presque rien : il traduit votre commande en requête HTTP vers `dockerd`, un **daemon** permanent, en **root**, qui fait tout le travail (construire, créer, stocker) et écoute sur une *socket* Unix, `/var/run/docker.sock`.

Podman n'a **pas de daemon**. Chaque commande `podman` est un programme ordinaire qui fait le travail lui-même, puis se termine ; le conteneur survit grâce à un minuscule superviseur, `conmon`, qui reste attaché à lui. Et surtout, Podman tourne par défaut en **rootless** : c'est *votre* utilisateur qui lance le conteneur, sans privilège. Le `root` que vous verrez dans le conteneur est une illusion du namespace `user` : côté hôte, c'est vous.

PODMAN

Pourquoi un second outil ? Deux reproches faits à Docker par les équipes d'exploitation : **un daemon root permanent** (un seul point de défaillance, et « qui peut parler à la socket est root ») et **une licence** (Docker Desktop est payant en entreprise depuis 2021). Podman répond aux deux : pas de daemon, rootless par défaut, gratuit. Et il a fait un choix décisif : sa CLI est **identique** à celle de Docker. `alias docker=podman` suffit dans 95 % des cas ; les images, les Dockerfiles et les registries sont les mêmes, parce que tout cela est OCI. Vous apprenez donc *Docker* — le vocabulaire de l'entreprise — avec Podman comme moteur.

Les deux partagent le reste : le **registry**, dépôt distant d'images (Docker Hub par défaut ; en entreprise Harbor, Nexus, GitLab Registry, ECR, ACR), et le **runtime** bas niveau (`runc` ou `crun`), qui demande réellement au noyau de créer les namespaces. Son nom apparaît dans les messages d'erreur.

Deux conséquences pratiques à comprendre tout de suite :

1. **Chez Docker, le travail se fait côté daemon.** Un chemin monté avec `-v /data:/data` est résolu sur le disque du *daemon*, pas du client — invisible en local, source de la moitié des surprises contre un daemon distant. Chez Podman, client et moteur sont le même processus.
2. **Accès au daemon Docker = accès root sur l'hôte.** Qui peut écrire sur `/var/run/docker.sock` peut lancer un conteneur privilégié et prendre la machine.

→ SÉCURITÉ

L'appartenance au groupe `docker` équivaut à un `sudo` sans mot de passe ni trace d'audit. Podman rootless est la réponse structurelle : un conteneur compromis n'a que *vos* droits.

5. Anatomie d'une commande

La CLI suit une grammaire régulière, la même pour les deux outils :

```
podman [objet] [action] [options] [cible] [arguments]
```

```
podman container run -d --name api -p 8080:8080 docker.io/library/eclipse-temurin:21-jre java -
version
#      _objet_ _action_ _options_ _image_ _
commande _
```

Les objets principaux sont `image`, `container`, `volume`, `network`, `system` (et `pod`, propre à Podman). Pour les opérations fréquentes, il existe des **raccourcis historiques** où l'objet est implicite — ce sont eux qu'on lit partout :

Forme complète	Raccourci courant
<code>podman container run</code>	<code>podman run</code>
<code>podman container ls</code>	<code>podman ps</code>
<code>podman image ls</code>	<code>podman images</code>
<code>podman image pull</code>	<code>podman pull</code>
<code>podman container rm</code> / <code>podman image rm</code>	<code>podman rm</code> / <code>podman rmi</code>

Trois commandes de diagnostic à connaître dès maintenant :

```
podman version  # version du client (et du serveur, s'il y en a un)
podman info     # état du moteur : rootless ?, cgroups, réseau, stockage, noyau
podman inspect  # toutes les métadonnées d'un objet, en JSON
```

PIÈGE

Avec Docker, `docker version` affiche deux blocs, *Client* et *Server* ; si le second manque, le daemon ne tourne pas ou vous n'avez pas le droit de lui parler — le problème n'est jamais dans votre commande. Avec Podman, il n'y a qu'un bloc : il n'y a pas de serveur. Le message « Cannot connect to the Docker daemon » que vous trouverez dans toutes les FAQ n'existe donc pas... sauf si vous utilisez `podman --remote` ou `podman machine`.

6. En entreprise

Sur une stack Spring Boot + Angular + PostgreSQL :

- Le back Spring Boot devient **une image** contenant un JRE et le JAR. La même image, au **digest** près, part sur l'intégration, la recette et la production : c'est la fin du « ça marche sur ma machine ».
 - Le front Angular est *buildé* (`ng build`) puis le résultat statique est embarqué dans une image nginx. Node ne survit pas à la production — labo 05.
 - PostgreSQL est tiré d'une image publique officielle ; on ne l'écrit pas, on la configure.
 - Ces trois images vivent dans un **registry privé**, poussées par la CI. Sur les serveurs, elles tournent avec Docker, Podman ou Kubernetes : l'image ne sait pas qui la lance, et c'est le but.
-

À retenir

- Un conteneur est un **processus isolé** par les *namespaces* et limité par les *cgroups*, exécuté par le **noyau de l'hôte** — pas une mini-VM. Sous Windows, ce noyau est celui de WSL 2.
- Une **image** est un modèle immuable ; un **conteneur** en est une instance vivante avec une couche d'écriture jetable.
- Docker = client + daemon root permanent ; Podman = un programme sans daemon, rootless par défaut. Même CLI, mêmes images, mêmes registries (OCI).
- Les conteneurs sont **jetable**s : tout état non externalisé disparaît à leur suppression.
- Accès au daemon Docker = accès root ; Podman rootless n'accorde que vos droits.
- La CLI suit `podman <objet> <action>` ; les formes courtes (`ps` , `run` , `images`) sont des raccourcis.

Vocabulaire

image : modèle immuable, empilement de couches en lecture seule. — **conteneur** : instance en exécution d'une image. — **layer** (couche) : fragment de système de fichiers issu d'une étape de construction. — **registry** : serveur qui stocke et distribue les images. — **repository** : ensemble des versions d'une même image (`postgres`). — **tag** : étiquette d'une version (`postgres:16-alpine`). — **daemon** : service permanent (`dockerd`) ; absent chez Podman. — **rootless** : mode où le moteur et les conteneurs tournent sous votre utilisateur. — **namespace** : isolation de la vue d'une ressource par le noyau. — **cgroup** : limitation de la consommation. — **runtime** : `runc` / `crun` , le composant qui crée réellement le conteneur. — **conmon** : petit superviseur de Podman attaché à chaque conteneur. — **OCI** : standard ouvert des images et des runtimes.